

Архитектурная эволюция видеоигровых бенчмарков: движковые инновации, системная оптимизация и вычислительные компромиссы

Графические архитектуры: эволюция растеризации, кластеризации света и аппаратной трассировки

Эволюция графических технологий в игровой индустрии представляет собой непрерывный переход от статических методов предварительных вычислений к динамической генерации оптических явлений непосредственно на графическом процессоре. Каждый технологический скачок диктовался необходимостью преодоления физических барьеров: дефицита пропускной способности оперативной памяти, ограничений скорости заполнения сцены пикселями (fill-rate) и пределов параллелизма вычислительных блоков.

Оригинальный проект *Doom* (1993, движок id Tech 1) продемонстрировал смену фундаментальной парадигмы в задаче упорядочивания геометрии по глубине в условиях жесткого дефицита ресурсов процессоров Intel 80386 и 80486. Классический Z-буфер требовал недостижимого для 4 МБ оперативной памяти объема под буфер глубины, а попиксельная сортировка в реальном времени приводила бы к мгновенному падению кадровой частоты.

Джон Кармак применил математический аппарат двоичного разбиения пространства (Binary Space Partitioning, BSP), опираясь на алгоритмические наработки Фукса, Кедема и Нейлора. При компиляции уровня внешняя утилита заранее выстраивала двумерное BSP-дерево секторов, что позволяло во время рендеринга обходить сцену от ближайших плоскостей к дальним (front-to-back) без единой операции сортировки в кадре. Стены выводились вертикальными экранными колонками с проверкой перекрытия через одномерный массив диапазонов, а пол и потолок заливались через алгоритм flood-fill в единые горизонтальные плоскости видимости (visplanes) с использованием целочисленной арифметики с фиксированной точкой 16.16.

Архитектурной платой за скорость стала невозможность создания честной трехмерной геометрии произвольной формы: карты оставались двумерными планами с переменными высотами потолков, исключая комнаты друг над другом и наклоны камеры по вертикали, а жесткий лимит в 128 одновременно видимых visplanes приводил к аварийному завершению работы при малейшей ошибке левел-дизайна.

По мере насыщения сцен динамическими источниками света индустрия в середине 2000-х годов столкнулась с квадратичной сложностью прямого рендеринга (Forward Rendering): вычислительные затраты масштабировались как $O(N \times M)$, где N — число

полигональных фрагментов, а M — число источников освещения. Это вызывало катастрофический перерасход скорости заполнения из-за затенения невидимых пикселей. Ответом стал переход к отложенному рендерингу (Deferred Shading). В *Metal Gear Solid V: The Phantom Pain* (Fox Engine, 2015) студия Kojima Productions довела разделение геометрии и освещения до технологического совершенства. Свойства сцены за один проход упаковывались в промежуточный G-буфер, после чего источники света вычислялись как двухмерные экранные объемы со сложностью $O(N + M)$. Чтобы снизить нагрузку на полосу пропускания памяти, разработчики отказались от тяжелых кубических карт для диффузного глобального освещения в пользу локальных зондов на базе сферических гармоник (Spherical Harmonics) второго порядка, упакованных всего в 9 коэффициентов на цветовой канал. Диффузная составляющая рассчитывалась в буфере половинного разрешения с последующим билатеральным апскейлингом с учетом карты глубин, что позволило достичь стабильных 60 кадров/с на базовой PlayStation 4.

Главными недостатками схемы оставались несовместимость с аппаратным сглаживанием MSAA и крайне затратная обработка полупрозрачных поверхностей, требовавшая параллельного форвардного прохода.

В 2020 году id Software в *DOOM Eternal* (движок idTech 7) совершила контринтуитивный вираж, полностью отказавшись от классического G-буфера в пользу кластеризованного прямого рендеринга (Clustered Forward Rendering). Причиной отказа стали неприемлемые накладные расходы видеопамати под буферы геометрии в разрешении 4K (свыше 250 МБ) и необходимость отрисовки сотен полупрозрачных эффектов частиц.

Архитектура idTech 7 под API Vulkan опирается на глубокий предварительный проход глубины (Depth Pre-pass) для раннего отсека невидимых фрагментов. Пространство усеченной пирамиды видимости разбивается на трехмерные кластеры, внутри которых специализированный compute-шейдер растеризует границы источников света в тайлы с упаковкой индексов в 64-битные битовые маски. Форвардный шейдинг обращается к плоскому массиву видимых источников через битовые операции без обращений к тяжелым буферам памяти.

Внедрение Bindless-текстур и универсальных Uber-шейдеров сократило число уникальных состояний графического конвейера (PSO) до рекордных ~500 штук на всю игру с динамическим слиянием вызовов отрисовки в единые косвенные буферы команд. Минусом такого подхода стал предельно высокий порог входа: алгоритм требует безупречного ручного управления барьерами синхронизации памяти GPU.

Растровые аппроксимации исчерпали потенциал с переходом к физически корректному переносу фотонов. Первым бескомпромиссным проектом эпохи аппаратного Ray Tracing стал *Metro Exodus: Enhanced Edition* (4A Engine, 2021), где разработчики полностью удалили растровые источники и запеченные карты света, сделав аппаратные RT-ядра обязательным требованием.

Основой пайплайна стала система диффузных зондов (DDGI), скомбинированная с темпоральным накоплением. Зонды зондируют сцену лучами в мировом пространстве, а результаты трассировки усредняются во времени, моделируя бесконечное число диффузных переотражений. Налогом на физическую честность стал эффект задержки распространения света: при резком включении мощного источника в темноте фотоны заполняют комнату с запаздыванием в 15–20 кадров, сопровождаясь высокочастотным шумом.

Вершиной трассировки стала интеграция трассировки путей (Path Tracing) на базе алгоритмов резервуарного пространственно-временного сэмплирования важности

(ReSTIR) в *Cyberpunk 2077* (режим RT Overdrive) и *Black Myth: Wukong*.

Алгоритм генерирует всего 1–2 луча на пиксель, после чего объединяет их с соседними пикселями в пространстве и предыдущими кадрами во времени через структуры резервуаров с весами правдоподобия. Для устранения расхождения лучей внутри вычислительных блоков GPU задействуется технология Shader Execution Reordering (SER), динамически перегруппировывающая расходящиеся потоки инструкций в когерентные варпы.

Внедрение реконструкции лучей (Ray Reconstruction) заменило отдельные пространственные фильтры для отражений и теней единой нейросетью, восстанавливающей высокочастотные детали из шумового сигнала.

В *Alan Wake 2* (Northlight Engine) к этому добавился полный отказ от устаревшего геометрического конвейера вершинных и индексных буферов в пользу сеточных шейдеров (Mesh Shaders). Это исключило фиксацию топологии на этапе компиляции, передав отсечение невидимых полигонов и выбор уровня детализации (LOD) на шейдеры задач и сеток, что сделало запуск игры технически невозможным на видеокартах без поддержки DirectX 12 Ultimate.

Проект	Движок / API	Базовая концепция света	Архитектурные преимущества	Критические системные ограничения
Doom (1993)	id Tech 1 / DOS	BSP front-to-back, 1D-буфер стен, заливка visplanes	Отсутствие сортировки и z-буфера, минимальные требования к CPU	Отсутствие многоэтажности, лимит 128 visplanes, плоская геометрия
MGS V (2015)	Fox Engine / DX11	Deferred Shading + SH-зонды + Half-res Bilateral Upsampling	Линейная сложность $O(N+M)$, сверхнизкое потребление VRAM под диффуз	Сложность полупрозрачных эффектов, отсутствие аппаратного MSAA
DOOM Eternal (2020)	idTech 7 / Vulkan	Clustered Forward + Depth Pre-pass + Bindless Uber-shaders	Отсутствие расхода VRAM под G-буфер, нативная прозрачность, сотни FPS	Экстремальная сложность синхронизации, зависимость от GPU compute
Metro Exodus EE (2021)	4A Engine / DX12, Vulkan	DDGI Infinite Bounces + Temporal Radiance Cache	Полный отказ от запекания, кинематографичный бесконечный отскок света	Не работает без аппаратного RT, темпоральный лаг распространения света
Cyberpunk 2077 (2023)	REDengine 4 / DX12	Full Path Tracing + ReSTIR GI/DI + SER + Ray	Корректный перенос фотонов от миллионов	Неиграбелен без апскейлинга и генерации

Проект	Движок / API	Базовая концепция света	Архитектурные преимущества	Критические системные ограничения
		Reconstruction	неоновых источников	кадров, шум в динамике
Alan Wake 2 (2023)	Northlight / DX12	Path Tracing + Mesh Shaders Geometry Pipeline	Идеальный микрополигональный каллинг, высокоточный рассеянный свет	Полный отказ от старых поколений GPU, тяжелая трассировка полупрозрачности

Стриминг открытых миров, иерархическая видимость и преодоление пропускной способности носителей

Создание бесшовных трехмерных пространств исторически упирается не в производительность исполнительных блоков GPU, а в пропускную способность шины ввода-вывода и время случайного доступа накопителя данных.

В 2001 году релиз *Grand Theft Auto III* на PlayStation 2 ознаменовал рождение концепции бесшовного открытого мира. Разработчики были зажаты между 32 МБ объединенной оперативной памяти консоли и медленным DVD-приводом со скоростью чтения 2–4 МБ/с и чудовищным временем позиционирования считывающей головки лазера. Загрузить 130 МБ ассетов города целиком было физически невозможно.

Команда DMA Design разделила виртуальный город на сектора и развернула вокруг камеры скользящее окно стриминга, подгружавшее геометрию спереди и выгружавшее ее сзади. Чтобы избежать фатальной фрагментации памяти, ведущий программист Оббе Вермей внедрил кастомный аллокатор: сотни мелких ассетов паковались в неизменяемые бинарные блоки с шагом ровно в 2 КБ, а физическая раскладка секторов на дорожках DVD строго зеркалировала пространственную топологию островов.

Физические ограничения носителя напрямую определили геймдизайн. Максимальная скорость автомобилей в коде была жестко ограничена, а в физику транспорта добавили скрытое сопротивление воздуха (+5% на высоких скоростях), чтобы игрок не мог обогнать упреждающий стриминг с оптического диска.

Планировка улиц Портленда подчинялась таймингам считывания: небоскреб посреди шоссе в районе Сент-Маркс был возведен исключительно ради того, чтобы заставить игрока притормозить и повернуть, предоставив DVD-приводу дополнительные полторы секунды на загрузку чанка. Полеты были заблокированы, так как скоростное перемещение по воздуху мгновенно опустошало буферы и приводило к провалу сквозь мир, а пул активного трафика ограничивался восемью моделями машин одновременно.

В *GTA: San Andreas* (2004) разработчики выжали ядро RenderWare до предела, выделив через конфигурационный файл stream.ini всего 13.5 МБ памяти под потоковую загрузку.

Полный отказ от разделения на острова был достигнут с помощью геймдизайнерского трюка: гигантские пустые пространства лесов и пустынь выполняли роль географического буфера памяти. Пока игрок пересекал сельскую местность, предыдущий мегаполис успевал полностью очиститься из памяти, а следующий — поблочно загрузиться.

Аналогичные принципы виртуализации оптического носителя использовались еще в *Crash*

Bandicoot (1996) для PlayStation 1. Naughty Dog разработала упаковщик NPT, разбиравший 8–16 МБ данных уровня на чанки по 64 КБ, непрерывно транслировавшиеся на скорости 300 КБ/с прямо в 2 МБ RAM с жестким сокрытием геометрии за поворотами рельсовой камеры.

С переходом к гигантским детализированным ландшафтам потоковый стриминг уперся в накладные расходы центрального процессора на регистрацию вызовов отрисовки. В движке Decima Engine (*Horizon Zero Dawn*, *Death Stranding*) студия Guerrilla Games перенесла управление видимостью и выбор уровней детализации (LOD) на вычислительные шейдеры GPU. Геометрия сцены организована в пространственные иерархические деревья ограничивающих объемов (BVH). Вычислительный конвейер строит пирамиду глубин предыдущего кадра (Hi-Z), после чего compute-шейдер выполняет консервативную проверку пересечения ограничивающих параллелепипедов объектов с буфером глубины, отсекая невидимые элементы с субпиксельной точностью до их попадания в растровый конвейер.

Для отрисовки ландшафтов применяется динамический клипмэппинг (Clipmapping) со смещением центра детализации: сетка земли генерируется концентрическими кольцами вокруг камеры с падением плотности треугольников по логарифмической шкале, а дальние пропсы сливаются в единые иерархические HLOD-кластеры.

В современных проектах стриминг сталкивается с дилеммой "задержка I/O против вычислительного бюджета декомпрессии". В *S.T.A.L.K.E.R. 2: Heart of Chornobyl* (Unreal Engine 5.1) и *Starfield* даже сверхбыстрые накопители стандарта PCIe 4.0 NVMe упираются в узкие места программной декомпрессии на центральном процессоре. Необходимость декодирования тысяч мелких виртуальных текстур и микрополигональной геометрии Nanite в реальном времени приводит к резким скачкам времени кадра (traversal stuttering) при пересечении границ секторов.

Проект / Движок	Архитектура стриминга	Бюджет оперативной памяти под мир	Механизм маскировки загрузки	Накладываемые ограничения
GTA III (RenderWare)	Скользящее окно, сектора фиксированного размера (2 КБ)	Около 16–20 МБ из 32 МБ PS2 RAM	Планировка улиц (слепые углы, здания-заслонки), погодный туман	Лимит скорости машин, запрет полетов, клоны автомобилей в трафике
GTA: San Andreas (RenderWare)	Топологический стриминг по зонам	13.5 МБ выделенной памяти (stream.ini)	Сельские буферные зоны между городами, туман горизонта	Потолок детализации движка, агрессивный pop-in текстур на скорости
Horizon Zero Dawn (Decima)	GPU-driven BVH culling, тороидальные клипмэпы	~4–5 ГБ объединенной памяти GDDR5 PS4	Иерархические HLOD-импостеры, динамический туман	Зависимость от латентности compute-очереди, шиммеринг вдалеке
Red Dead Redemption 2 (RAGE)	Потоковый terrain clipmapping + асинхронный	До 6–8 ГБ системной и видеопамяти на	Пересеченный рельеф, кинематографиче	Высокая нагрузка на CPU при распаковке,

Проект / Движок	Архитектура стриминга	Бюджет оперативной памяти под мир	Механизм маскировки загрузки	Накладываемые ограничения
	ввод-вывод	ПК	ская камера, волюметрия	требование к многопоточности

Системная симуляция: физика субстанций, химические движки и вычислительные бюджеты ИИ

Интеграция глубокой симуляции в игровой процесс радикально меняет структуру времени кадра, перенося вычислительный центр тяжести с графического процессора на ядра центрального процессора.

До выхода *Half-Life 2* (2004) физика в играх носила преимущественно декоративный характер. Инженеры Valve лицензировали middleware Havok 2, но переписали игровой слой, создав концепцию "Physics AI" и сквозную систему субстанций (Substance System). Внутри движка Source каждый объект определялся исключительно параметрами физического материала: массой, плотностью, коэффициентом трения, плавучестью и хрупкостью, которые считывались из конфигурационных таблиц.

На этой базе была реализована гравитационная пушка, превратившая окружение в интерактивный арсенал. Для обеспечения математической стабильности движок использовал фиксированный шаг физической симуляции (fixed timestep physics, 66 Гц / 15 мс) с линейной интерполяцией положений тел между физическими тиками для рендеринга.

Для предотвращения лавинообразного роста числа проверок коллизий была реализована агрессивная спячка тел (Physics Sleeping): объект, чья кинетическая энергия падала ниже пороговой, исключался из контактного графа до получения внешнего импульса.

В *Portal* (2007) Valve модифицировала пространственную структуру Source, решив задачу разрыва физической непрерывности. Портал представляет собой виртуальную щель с рекурсивной подменой матрицы координат. При частичном пересечении портала движок создает дубликат коллизионного тела по другую сторону с сохранением вектора импульса, а рекурсивный рендеринг сквозь порталы маскируется через stencil-буфер, умножая число вызовов отрисовки в несколько раз.

В *The Legend of Zelda: Breath of the Wild* (2017) Nintendo развернула "химический движок" поверх модифицированного ядра Havok. Симуляция опирается на матрицу взаимодействия трех категорий: элементов (Огонь, Вода, Лед, Электричество, Ветер), материалов (Дерево, Металл, Камень) и набора нерушимых правил. Элемент трансформирует состояние материала (огонь сжигает дерево, порождая восходящий поток теплого воздуха), элементы взаимодействуют между собой (вода тушит огонь, испаряясь в пар), но материалы напрямую друг на друга не влияют.

Эта замкнутая система породила мультипликативный геймплей за счет спонтанной комбинаторики, однако слабейший трехъядерный процессор Espresso консоли Wii U захлебывался при масштабных физических взаимодействиях. Из-за двойной буферизации вертикальной синхронизации малейшее превышение бюджета времени кадра в 33.3 мс приводило к жесткому ступенчатому падению частоты кадров с 30 до фиксированных 20 кадров/с.

Парадигма искусственного интеллекта также претерпела слом. В *Left 4 Dead* (2008) Valve

и Turtle Rock заменили фиксированные скрипты мета-системой AI Director. Директор управлял драматическим пейсингом через три архитектурных слоя: расчет расстояния потока (Flow Distance) по графу навигационной сетки от старта к убежищу, отсечение зон вне зоны внимания игроков через активное множество областей (Active Area Set, AAS) с пулингом сущностей зараженных, и адаптивное регулирование стресса.

Директор отслеживал состояние выживших и циклически переключал фазы симуляции: от нагнетания орды (Build Up) и пиковой атаки (Sustain) к спаду спавна (Peak Fade) и обязательному периоду покоя (Relax) длительностью 30–45 секунд. Важно, что система динамически меняла плотность и частоту событий, никогда не искажая базовые параметры урона или точности врагов.

Попытка реализовать независимый ИИ для сотен горожан в *Baldur's Gate 3* (Divinity Engine 4.0) привела к тяжелейшему кризису производительности в третьем акте игры.

Центральные процессоры уровня AMD Ryzen 5 3600 теряли до 50% кадровой частоты во Вратах Балдура, поскольку движок рассчитывал навигацию, зрение, реакцию на преступления и диалоговые деревья для огромной толпы в едином координатном пространстве.

Масштабирование на многоядерных процессорах блокировалось межпоточной синхронизацией общих ресурсов памяти. Проблему удалось купировать только введением жесткого бюджета симуляции (agent_update_budget): дальние NPC переводились в режим упрощенных пространственных автоматов без физического поиска пути с обновлением анимаций с пониженной частотой.

Механика / Проект	Архитектура ядра	Алгоритмический базис	Вычислительная цена	Методы деградации под нагрузкой
Физика субстанций (<i>Half-Life 2</i>)	Havok 2 + Physics AI (Source)	Фиксированный шаг (66 Гц), интегратор Верле, лучевые коллизии	Высокая нагрузка на одиночное ядро CPU при массовых свалках	Перевод объектов в режим Physics Sleeping, упрощенные боксы коллизий
Химический движок (<i>Zelda: BotW</i>)	Проприетарный слой поверх Havok	Матрица конечных автоматов взаимодействия стихий и материалов	Всплески времени кадра при лавинообразных цепных реакциях	Пропуск шагов распространения пламени, падение FPS до фиксированных 20
AI Director (<i>Left 4 Dead</i>)	Иерархический мета-контроллер	Flow field tracking, функции стресса, пулинг сущностей	Минимальная нагрузка благодаря расчету исключительно внутри AAS	Полное отсечение симуляции невидимых зон, деспавн отставших врагов
Симуляция толпы (<i>Baldur's Gate 3</i>)	Divinity Engine 4.0 Task Graph	Поведенческие деревья + глобальный навигационный граф	Тяжелый CPU-bound, межпоточные блокировки структур данных	Введение Actor LOD: заморозка логики и пропуск кадров анимации дальних NPC

Сетевая синхронизация, протоколы ввода и

серверная инфраструктура

Сетевой код в интерактивных системах вынужден балансировать между фундаментальным физическим ограничением (скоростью света в оптическом волокне) и психологическим требованием мгновенного отклика на действия игрока.

В соревновательных шутерах стандартом являлась клиент-серверная модель с авторитарным сервером и клиентской компенсацией запаздывания (Lag Compensation). В *Counter-Strike: Global Offensive* (движок Source) состояние мира квантовалось жесткими тиками (64 Гц в подборе матчей, 128 Гц на киберспортивных турнирах). При выстреле сервер физически "перематывал" хитбоксы оппонентов назад во времени на величину сетевого пинга стрелка и буфера интерполяции (`cl_interp`), верифицировал попадание лучом и возвращал состояние назад.

Это гарантировало точность стрельбы, но порождало неустранимый артефакт "смерти за углом", когда скрывшийся за стеной игрок погибал из-за регистрации попадания в его прошлую пространственную проекцию на клиенте с высоким пингом.

В 2023 году Valve в *Counter-Strike 2* (Source 2) внедрила архитектуру Sub-Tick. Вместо квантования действий строго по границам тика клиент снабжает каждый клик высокоточным таймстемпом внутри текущего кадра. Сервер рассчитывает баллистику ровно в тот субмомент времени, когда было совершено нажатие, отвязав расчет попадания от частоты дискретизации.

Однако снапшоты состояния мира продолжают рассылаться с базовой частотой 64 Гц, что вызвало визуальную рассинхронизацию на старте: регистрация выстрела сервером происходила между тиками, а анимация попадания и рэгдолла проигрывалась только со следующим снапшотом, порождая ощущение невнятного отклика.

Принципиально иной путь выбрала Blizzard в *Overwatch*, построив сетевое ядро вокруг детерминированной симуляции на чистом ECS. Архитектура закрепила принцип "Favor the Shooter" ("Преимущество стреляющему"): сервер безоговорочно валидирует попадание снарядами прямого действия (hitscan), если на экране атакующего цель находилась в перекрестии.

Исключение сделано лишь для способностей абсолютной неуязвимости (рывок Трейсер, криостазис Мэй), имеющих высший сетевой приоритет. Переход с формата 6v6 на 5v5 в *Overwatch 2* диктовался не только геймдизайном, но и сетевой экономикой: симуляция двенадцати персонажей со сложными аурами, щитами и контурами перегружала бюджеты кадров и раздувала сетевые снапшоты, вызывая просадки тикрейта ниже критических 120–144 Гц.

В файтингах задержка даже в один кадр (16.6 мс) ломает микротайминги комбинаций. Катастрофическим примером собственной реализации стал запуск *Street Fighter V* (2016), где проприетарный rollback-код Capcom страдал от десинхронизации внутренних часов клиентов (Clock Drift). Если один из компьютеров проседал по частоте кадров, игра накапливала асимметричную задержку, порождая лавинообразные телепортации персонажей (one-sided rollback) для одного из участников.

Эталоном проектирования стал *Guilty Gear Strive* (2021, Arc System Works). Сетевой стек создавался параллельно с геймплеем на Unreal Engine 4 с полным разделением игрового состояния и визуального представления. Детерминированная логика (хитбоксы, здоровье, шкалы) полностью изолирована от тяжелых объектов (частицы, скелеты, шейдеры). При получении запоздавшего пакета движок мгновенно откатывает стейт мира на несколько кадров назад, накатывает ввод, симулирует пропущенные тики физики за доли

миллисекунды и перерисовывает кадр.

Чтобы процессор базовой PlayStation 4 не захлебывался при многократной симуляции скелетных анимаций за кадр, инженеры переписали низкоуровневый код трансформации костей под векторные регистры SIMD, добившись безупречного сетевого отклика даже при пинге свыше 120–150 мс.

В многопользовательских мирах барьером выступает квадратичная сложность рассылки пакетов:

$$P = O(N^2)$$

где N — число игроков в локации. Если при 20 игроках сервер генерирует 380 пакетов за тик, то при скоплении 1500 игроков объем широковещательных рассылок подскакивает до $1500 \times 1499 \approx 2,248,500$ пакетов за сетевой тик, вводя серверные ядра в состояние взаимной блокировки (deadlock).

World of Warcraft (Blizzard) преодолевала эту проблему на протяжении двух десятилетий.

Во время открытия Врат Ан'Киража в *WoW Classic* серверы выдерживали перегрузку за счет протоколов прогрессивной деградации точности: при критической плотности сервер отключал рассылку векторов вращения взгляда (facing-updates), превращая персонажей в синхронно смотрящих в одну точку манекенов, а второстепенные пакеты буферизировались в очереди.

Внедрение динамического шардинга (Sharding) и послойной изоляции (Layering) позволило прозрачно распределять игроков по изолированным копиям локаций с распределением нагрузки по кластерам. Системным компромиссом шардинга стало разрушение целостности мира: внезапное исчезновение врагов прямо во время боя, фантомные ноды ресурсов и манипуляции со слоями ради фарма.

Проект	Сетевая модель	Архитектурная парадигма	Поведение при пинге и потере пакетов	Системная цена и артефакты
CS:GO (2012)	Клиент-сервер, дискретный тик (64/128 Гц)	Авторитарный сервер, Lag Compensation, перемотка хитбоксов	Клиентская интерполяция, экстраполяция сущностей	"Смерть за углом", налог на CPU сервера при 128 тиках
CS2 (2023)	Клиент-сервер, Sub-tick внутри 64 Гц базы	Высокоточные таймстемпы действий, непрерывная баллистика	Регистрация точки выстрела в точный субмомент ввода	Визуальная рассинхронизация рендера и подтверждения попадания
Overwatch (2016)	Клиент-сервер на базе ECS, детерминизм	"Favor the Shooter", приоритет способностей неуязвимости	Агрессивная экстраполяция векторов, сглаживание дельт	Смерти в неуязвимости из-за латентности, ограничение пула игроков
Guilty Gear Strive (2021)	Peer-to-Peer, детерминированный Rollback	Изоляция игрового состояния, фоновая перемотка на K	Мгновенный локальный отклик, откат и быстрая ресимуляция	Высокая нагрузка на CPU при пересчете нескольких тиков за кадр

Проект	Сетевая модель	Архитектурная парадигма	Поведение при пинге и потере пакетов	Системная цена и артефакты
		кадров		
World of Warcraft (2004–2024)	Распределенный серверный кластер	Динамический шардинг, пространственный лееринг	Снижение тикрейта зоны, отключение второстепенных апдейтов	Разрыв групп, десинхронизация слоев, визуальное исчезновение сущностей

Системная инженерия ПК-портов, низкоуровневые API и аппаратные барьеры

Переход к низкоуровневым API (DirectX 12, Vulkan) ознаменовал радикальную трансформацию: управление видеопамятью, барьерами синхронизации конвейера и компиляцией программного кода было снято с драйверов видеокарт и переложено целиком на разработчиков игрового движка.

В API предыдущих поколений (DirectX 11) графический драйвер самостоятельно компилировал шейдеры под целевую архитектуру GPU и кэшировал их в фоновом режиме, маскируя всплески времени кадра.

В DirectX 12 появилась монолитная концепция Pipeline State Object (PSO): шейдеры, форматы буферов, режимы смешивания, тест трафарета и топология примитивов должны быть скомпилированы в единый неизменяемый бинарный объект строго до вызова команды отрисовки. Число комбинаций PSO в крупных проектах достигает сотен тысяч. Если игра пытается скомпилировать отсутствующий PSO в реальном времени (Just-In-Time компиляция), графический поток CPU блокируется на 30–250 мс, вызывая тяжелый компиляционный статтер (shader stuttering).

Катастрофическим проявлением этой архитектурной проблемы стал релиз ПК-версии *The Last of Us Part I* (2023, порт Iron Galaxy и Naughty Dog). Движок, спроектированный под фиксированную архитектуру PlayStation 5 с готовыми бинарными шейдерами, попытался провести компиляцию сотен тысяч PSO в фоновом режиме при первом запуске.

Это привело к стопроцентной утилизации всех процессорных ядер в течение нескольких часов, утечкам системной памяти, OOM-крашам и сбоям стриминга ассетов. Лишь в последующих патчах разработчики смогли реструктурировать размер блоков PSO-кэша и переработать стриминг текстур.

Схожий кризис пережил проект *Elden Ring* (2022, FromSoftware), где отсутствие предварительного прогрева конвейера (PSO warmup) и лавинообразное порождение буферов команд приводили к зависаниям при каждом новом спецэффекте. На консоли Steam Deck инженеры Valve решили эту проблему на системном уровне, распространяя предварительно скомпилированный монолитный кэш шейдеров через Steam прямо на устройства пользователей.

В конце 2000-х годов индустрия столкнулась с крушением надежд на бесконечный рост тактовой частоты процессоров, которая уперлась в тепловой барьер на уровне ~3.5–4 ГГц, вынудив производителей перейти к экстенсивному наращиванию параллельных ядер. Проекты, архитектура которых опиралась на один доминирующий мастер-поток, оказались в ловушке.

ПК-релиз *Grand Theft Auto IV* (2008, движок RAGE) сочетал процедурную симуляцию Euphoria и отложенный рендеринг, однако главный цикл обработки мира и трансляция вызовов отрисовки через DirectX 9 замыкались на одном основном процессорном ядре. Видеокарты простаивали с половинной загрузкой, пока первое ядро процессора захлебывалось в непрерывном троттлинге.

Этот архитектурный дефект непреодолим временем: даже современные чипы с рекордной однопоточной производительностью (AMD Ryzen 7 9800X3D) демонстрируют нестабильный график времени кадра, а попытки внедрить трассировку путей через RTX Remix упираются в барьер 40 кадров/с на флагманских RTX 5090 исключительно из-за очередей шестнадцатилетнего однопоточного кода.

В *Forza Horizon 3* (2016) команда Playground Games наступила на те же грабли, утилизировав нулевое ядро CPU на 100%, пока остальные ядра оставались ненагруженными. Лишь в *Forza Horizon 4* (2018) движок ForzaTech подвергся фундаментальной перестройке с внедрением модели параллельных независимых вычислительных задач (Task-Based Job System), что обеспечило идеальное масштабирование нагрузки по всем ядрам и позволило получить стабильные 60 кадров/с даже на бюджетных процессорах.

В современном конвейере акцент сместился на алгоритмы темпоральной реконструкции и генерации кадров. В *Ghost of Tsushima* (2024, порт Nixxes) разработчики впервые развязали апскейлинг суперразрешения и генерацию кадров, позволив активировать AMD FSR 3.0 Frame Generation поверх базы NVIDIA DLSS Super Resolution.

При этом нативное сглаживание TAA страдало от мерцания субпиксельной листвы, в результате чего реконструкция из пониженного разрешения через DLSS обеспечивала более чистое и стабильное изображение, чем честный рендеринг в нативном 4K. Генераторы кадров, интерполирующие дополнительный кадр на основе векторов оптического потока, не снижают системную задержку ввода, а увеличивают ее из-за удержания кадров в буфере отображения, требуя обязательной интеграции протоколов низких задержек (NVIDIA Reflex, AMD Anti-Lag) для динамического выравнивания очередей команд.

Максимальные пресеты графики ("Ultra") демонстрируют эффект убывающей отдачи. В *Red Dead Redemption 2* ультра-пресеты снижают кадровую частоту на 40–50% относительно оптимизированных высоких настроек без визуального выигрыша: просчет маршрутирования лучей в волюметрическом тумане (Near/Far Volumetric Resolution) выполняется в полном разрешении вместо половинного с билатеральным размытием, отнимая до 5 мс времени кадра, а включение MSAA поверх отложенного света съедает пропускную способность видеопамати.

Проект	Движок / API	Корневая инженерная проблема	Последствия на релизе	Архитектурное исправление
GTA IV (2008)	RAGE / DX9	Линейный рендер-поток, однопоточный CPU-bound цикл	Статтеры, недогруз GPU, неиграбельность на мощных ПК	Архитектурно не исправлено; частичный обход через грубую мощь CPU
Batman: Arkham City (2011)	UE3 / DX11	Нестабильный ранний DX11, неоптимизирован	Вылеты на 32-битных ОС, статтеры	Патчи очередей команд, перенос части расчетов

Проект	Движок / API	Корневая инженерная проблема	Последствия на релизе	Архитектурное исправление
		новый PhysX	тесселяции, просадки FPS	физики на CPU
Forza Horizon 3 (2016)	ForzaTech / DX12 (UWP)	Завязка на одно мастер-ядро, неэффективная аффинность	100% упор в Core 0, нестабильные 30–45 FPS на мощных GPU	Внедрение Out-of-order рендера, перестройка аффинности потоков
Elden Ring (2022)	Собственный / DX12	JIT-компиляция PSO, взрывной рост command buffers	Стоп-кадры до 250 мс при появлении новых спецэффектов	Частичные патчи; прекомпиляция монолитного кэша силами Valve
The Last of Us Part I (2023)	Собственный / DX12	Неуправляемый PSO-кэш, нехватка системных бюджетов	Многочасовая сборка кэша, OOM-краши, 100% нагрузка на CPU	Патчи: сжатие PSO-кэша, оптимизация текстурного стриминга
The Witcher 3 Next-Gen (2022)	REDengine 3 / DX12	DX12-враппер поверх DX11-кода, упор в CPU-потоки	Падение FPS на 45% в Новиграде без RT, статтеры, краши	Сохранение DX11-ветки; обход CPU-стены через Frame Generation

Архитектурные паттерны и системное влияние на индустрию

Технологический прогресс видеоигровых систем определяется не наращиванием грубого числа полигонов, а способностью архитектуры эффективно преодолевать физические барьеры аппаратных платформ.

Первая эпоха (1990-е — середина 2000-х годов) являлась эпохой жестких ограничений оперативной памяти и медленных физических носителей. Алгоритмы предварительного пространственного разбиения Джона Кармака в *Doom*, скользящие окна стриминга с кастомными блочными аллокаторами в *GTA III* и динамическая виртуализация компакт-диска в *Crash Bandicoot* доказали фундаментальный тезис: предельные рамки аппаратного обеспечения служат катализатором прорывных инженерных идей. Физическая латентность считывающей лазерной головки оптических приводов сформировала саму пространственную структуру открытых миров — от геометрии извилистых городских кварталов до маскирующих климатических барьеров.

Вторая эпоха (конец 2000-х — 2010-е годы) обнажила кризис однопоточного процессорного быстродействия и исчерпание возможностей прямого рендеринга. Иллюзия роста тактовой частоты ядер кремния разбилась о тепловой барьер, оставив проекты с монопольным главным циклом симуляции (*GTA IV*, *Crysis*) в статусе вечных технических аутсайдеров.

В то же время лавинообразное увеличение числа динамических источников света потребовало изоляции геометрического прохода от затенения через структуры G-буферов (MGS V). Студии, первыми внедрившие асинхронные графы задач и принципы детерминированной изоляции данных ECS (*Overwatch*, *Forza Horizon 4*), создали базис для индустриальных движков нового поколения.

Современная эпоха знаменует фундаментальный переход от детерминированной растровой геометрии и эвристических методов освещения к стохастическим симуляциям и машинному обучению. Переход к аппаратной трассировке путей через алгоритмы ReSTIR, тотальный перенос пространственной фильтрации на микросеточные конвейеры Mesh Shaders, а также замещение классических алгоритмов сглаживания обученными нейросетями реконструкции лучей окончательно превратили GPU в суперкомпьютер пространственно-временных вероятностных расчетов.

Успех современного интерактивного проекта определяет строгость управления ресурсами: способность архитектуры вовремя отсекал симуляцию невидимого пространства, квантовать физические шаги, детерминированно изолировать сетевое состояние и закладывать предварительный прогрев конвейерных состояний как базовое условие стабильности времени кадра.

Источники

1. Crysis has no business looking this good for a game released in 2007, https://www.reddit.com/r/gaming/comments/bmaz0a/crysis_has_no_business_looking_this_good_for_a/ 2. So how did Crysis manage to look so good for its time? : r/pcgaming, https://www.reddit.com/r/pcgaming/comments/1ds8ks2/so_how_did_crysis_manage_to_look_so_good_for_its/ 3. NSFW - Graphical Fidelity I Expect This Gen | Page 661 - NeoGAF, <https://www.neogaf.com/threads/graphical-fidelity-i-expect-this-gen.1623441/page-661> 4. Apple Announces 'Game Mode' in macOS Sonoma for Better Mac, https://www.reddit.com/r/apple/comments/141mna4/apple_announces_game_mode_in_macos_sonoma_for/ 5. r/Games on Reddit: Digital Foundry: Alan Wake 2 PC Path Tracing, https://www.reddit.com/r/Games/comments/17nnxc/digital_foundry_alan_wake_2_pc_path_tracing_the/ 6. Graphics Programming Weekly - Database - Jendrik Illner, https://www.jendrikillner.com/article_database/ 7. [MAJ] Alan Wake 2 nous fait peur avec ses configurations PC, <https://www.factornews.com/actualites/maj-alan-wake-2-nous-fait-peur-avec-ses-configurations-pc-49909.html> 8. Decima Engine: Visibility in Horizon Zero Dawn | PPTX - Slideshare, <https://www.slideshare.net/slideshow/decima-engine-visibility-in-horizon-zero-dawn/82371923> 9. Ever wonder how rendering works from your perspective? - Reddit, https://www.reddit.com/r/xboxone/comments/fmtx85/ever_wonder_how_rendering_works_from_your/ 10. Advances in Real-Time Rendering- SIGGRAPH 2017, <https://advances.realtimerendering.com/s2017/index.html> 11. News Posts matching 'Remedy' | TechPowerUp, <https://www.techpowerup.com/news-tags/Remedy> 12. Blog - The Toolsmiths, <https://thetoolsmiths.org/blog> 13. Opinion - Evolution of Graphics Technology | Page 2 - NeoGAF, <https://www.neogaf.com/threads/evolution-of-graphics-technology.1581569/page-2> 14. News Posts matching 'Remedy Entertainment' | TechPowerUp, <https://www.techpowerup.com/news-tags/Remedy%20Entertainment> 15. Technical links - Sly's homepage - Free, <http://tfpsly.free.fr/bookmarks.html> 16. Digital Foundry Direct Weekly - Acast, <https://feeds.acast.com/public/shows/69f9a6533a143563edc22e7f> 17. OTFCG/Awesome-Game-Analysis - GitHub,

<https://github.com/OTFCG/Awesome-Game-Analysis> 18. New Crysis vs Crysis Remastered Graphics Comparison : r/pcgaming,
https://www.reddit.com/r/pcgaming/comments/hkgn9i/new_crysis_vs_crysis_remastered_graphics/